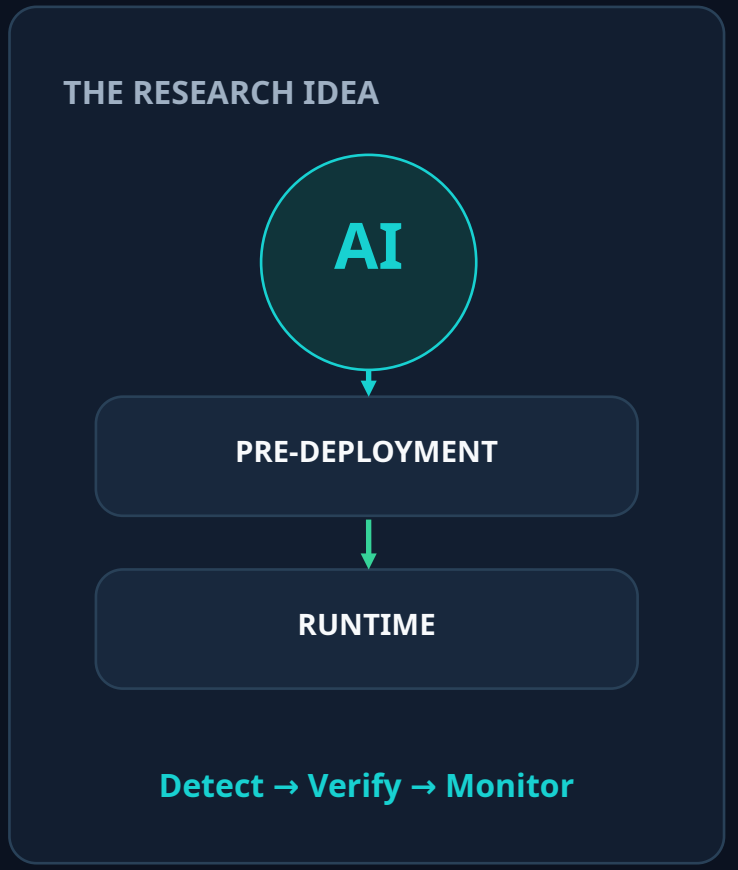
1ST RESEARCH MONITORING

AI IN CLOUD SECURITY

An Active AI Security Layer for Misconfiguration & Exposure Prevention in Cloud-Hosted Open-Source Software

Research with Honors • B.Tech CSE • JNEC

DOMAIN: AI IN CLOUD SECURITY | SCOPE: CLOUD OSS | STATUS: MONTH 1 ✓



Presenter: Manish Patil
Department of Computer Science & Engineering
1st Honors Monitoring Review

Academic framing: proposed research architecture • not a claimed production security product

What exactly is my research?

From a broad domain → to one testable research problem.

1 • DOMAIN

AI in Cloud Security

Broad field of study

2 • PROBLEM

Misconfiguration & Exposure

Specific risk boundary

3 • RESEARCH ARTIFACT

Active AI Security Layer

A proposed middleware that:

- audits IaC before deployment
- verifies suspicious network exposure
- monitors runtime telemetry

Research question: Can combining semantic AI auditing + active verification + runtime anomaly detection reduce non-actionable alerts without losing security coverage?

Title wording follows the finalized scope in the research notebook.

Why does this problem matter?

Cloud OSS is powerful, but configuration becomes part of the attack surface.

OPEN-SOURCE SOFTWARE → FAST DEPLOYMENT → CONFIGURATION DRIFT / ERROR → RISK

PORT EXPOSURE

e.g. database / cache reachable beyond intended boundary

PRE-DEPLOY

SECRET LEAKAGE

keys or tokens embedded in IaC / Docker / Helm

PRE-DEPLOY

OVER-PERMISSION

excessive RBAC or weak access control

PRE-DEPLOY

RUNTIME ABUSE

unexpected process / network behavior after deploy

POST-DEPLOY

Motivation for this research: security teams need evidence-backed prioritization, not just more alerts.

Literature repeatedly identifies misconfiguration, contextual uncertainty and alert noise as key challenges.

What is the exact research problem?

Three failure points create the research opportunity.

1 Static-only detection

Finds a possible problem in configuration, but may not know whether the risk is reachable or exploitable.

2 Rule-only verification

Can test known cases, but rigid rules struggle with custom or novel configuration semantics.

3 Runtime-only monitoring

Sees what happens after deployment, but can miss secrets and IaC mistakes before deployment.

Research problem → **How can we combine semantic understanding, live verification and runtime evidence into one lightweight security layer?**

The scope intentionally excludes "all cybersecurity"; it targets cloud-hosted OSS configuration and exposure risks.

What did the literature survey reveal?

The components exist individually; the research opportunity is in how they are combined and evaluated.

STATIC AI / RAG

SARGE

Semantic IaC auditing
RAG + LLM reasoning

ACTIVE VALIDATION

Dikshant & Verma

Behavioral probes
93% FP reduction reported

RUNTIME ML

Bajracharya et al.

Multi-telemetry
ML for K8s threats

MULTI-LAYER AI SEC.

AI-Infra-Guard

Layer-matched security
for AI infrastructure

SYNTHESIZED GAP

A lightweight, cloud-OSS-focused loop that con SEMANTIC AUDIT VERIFY MONITOR and measures the gain over baselines.

Academic novelty claim: **“Novel integration + controlled empirical validation within a defined OSS scope.”**

Current search also found AI-Infra-Guard (2026): use it as a related-work boundary, not as a claim that your idea has no precedent.

What already exists — and where does each stop?

The goal is not to replace all security tools; it is to close a specific workflow gap.

APPROACH	INTELLIGENCE	WHEN	STRENGTH	LIMITATION
Legacy CSPM / SAST	Rules / signatures	Pre-deploy	Strong for known patterns	Limited runtime context
Static RAG-LLM	LLM + retrieval	Pre-deploy	Semantic reasoning	No live reachability proof
Active Prober	Scripts / checks	Validation	Evidence of exposure	Rule dependency
Runtime ML	eBPF / logs + ML	Post-deploy	Behavior anomalies	Pre-deploy blind spot
Proposed layer	RAG + probes + runtime ML	End-to-end	Evidence + context	Must prove added value

Research position: complement existing tools with a closed feedback loop — not “replace everything.”

What is the proposed solution?

A dual-engine security wrapper with one evidence-based risk score.

ENGINE 1

Pre-Deployment Guard

- **IaC input** — Terraform • YAML • Dockerfile
- **RAG retrieval** — CIS • NVD • security knowledge
- **Local LLM** — Ollama + open model
- **Active validation** — TCP / HTTP reachability in staging

ENGINE 2

Runtime Telemetry Monitor

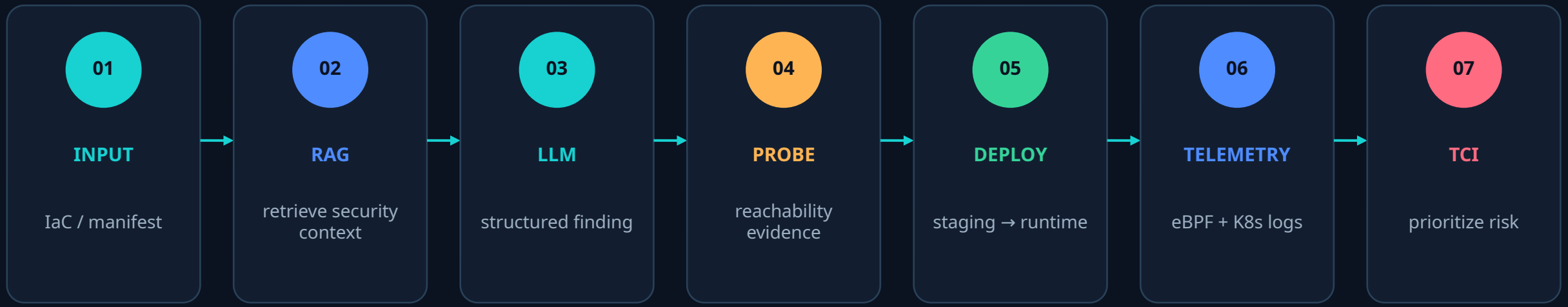
- **Telemetry** — eBPF + Kubernetes audit logs
- **Features** — Sliding event windows
- **Anomaly ML** — Isolation Forest / Autoencoder
- **Response** — High-confidence isolation policy

TCI = static risk × verification evidence + runtime anomaly + asset criticality

The "dual-engine" design is a proposed architecture; performance claims remain hypotheses until experimentally measured.

How will it actually work?

End-to-end pipeline: Scan → Retrieve → Reason → Verify → Deploy → Monitor → Score.



TCI (proposed)

$$w_1 \cdot S^{RAG} \cdot (1 - \delta \cdot P_{probe}) + w_2 \cdot A_{telemetry} + w_3 \cdot C_{asset}$$

All components normalized; weights sum to 1.0.

Design principle: AI suggests • verification supplies evidence • telemetry supplies runtime context

For the first monitoring, present TCI as a proposed formalization; do not present weights or δ as experimentally proven yet.

How will I implement it in 8–9 months?

A staged plan reduces technical risk and gives a measurable milestone every month.

M1	M2	M3–4	M5–6	M7	M8–9
LITERATURE + GAP	**TESTBED + DATA + RAG**	**ENGINE 1**	**ENGINE 2**	**INTEGRATION**	**PAPER + DEFENSE**
✓ Complete	● Active	RAG + probes	eBPF + anomaly ML	red-team + benchmark	manuscript + thesis

CURRENT STATUS

Month 1 complete | **Month 2 active** | **Local / open-source testbed**

Target testbed: Redis • PostgreSQL • Nginx • MinIO | Docker / k3s | ChromaDB | Ollama | Falco/Tetragon

A local testbed keeps the project reproducible and minimizes cloud cost.

How will I prove that it works?

Success is defined by measurable improvement against clear baselines.

FALSE-POSITIVE REDUCTION	**Target ≥ 90%**	compare alerts before vs. after verification
F1 SCORE	**Target ≥ 0.92**	precision/recall on labeled benchmark cases
AUC-ROC	**Target ≥ 0.95**	runtime anomaly discrimination
LATENCY	**Target < 3.5 s**	per static manifest; probe < 2 s
OVERHEAD	**Target < 5% CPU**	runtime telemetry footprint

Compare: baseline scanner → RAG only → RAG + probe → full hybrid

Targets are research hypotheses/benchmarks; they are not results yet.

What have I actually completed so far?

First monitoring should show evidence of progress—not pretend the prototype is finished.

DONE — Research domain + title fixed

DONE — Literature survey + synthesized gap

DONE — System boundary + architecture draft

DONE — TCI mathematical formulation

ACTIVE — Local testbed + dataset + RAG index

NEXT — Engine 1 implementation

Immediate deliverables: reproducible testbed • labeled corpus • retrieval demo • baseline measurements

Status wording is intentionally conservative: proposed architecture + active implementation, not completed production deployment.

Thank you • Questions & discussion

Use the answer pattern: definition → why → simple example → research relevance.

WHAT

What does the component mean?

Define it in one sentence.

WHY

Why is it needed here?

Connect it to the research gap.

HOW

How will you prove it?

Point to architecture, baseline and metrics.

“I am testing a hypothesis; the experiment will determine the final numbers.”

Thank you

Backup: Threat Composite Index (TCI)

Use only when a panel member asks for the mathematical detail.

$$TCI(a,t) = w_1 \cdot S^{RAG}(a) \cdot (1 - \delta \cdot P_{probe}(a)) + w_2 \cdot A_{telemetry}(a,t) + w_3 \cdot C_{asset}(a)$$

- S^{RAG} — static risk from RAG audit
- P_{probe} — 0/1 reachability evidence
- $A_{telemetry}$ — runtime anomaly score
- C_{asset} — asset criticality
- w_1, w_2, w_3 — weights; sum = 1

Important: δ is literature-inspired; estimate and report the value from your own data.

Backup: Dataset, ground truth & baselines

This is where you demonstrate that the research is experimentally testable.

STATIC DATA

Clean + vulnerable IaC / Docker / Helm

- public benchmark repos
- seeded misconfigurations
- labels by vulnerability type

KNOWLEDGE BASE

Security evidence for RAG

- CIS benchmarks
- NVD CVE records
- controlled policy guidance

RUNTIME DATA

Benign baseline + controlled attacks

- normal workloads
- Atomic Red Team / safe simulations
- system / K8s telemetry

Ground truth = known labels + controlled reachability checks + annotated runtime events

Key literature & technical foundations

Use these citations in the thesis; keep the slide readable by grouping them.

1. Goyal, M. K., & Chaturvedi, R. “Detecting Cloud Misconfigurations with RAG and Intelligent Agents.” Journal of Electrical Systems, 20(11s), 2558–2570. DOI: 10.52783/jes.7882.

2. Dikshant, & Verma, G. “Reducing False Positives with Active Behavioral Analysis for Cloud Security.” arXiv:2508.12584 / later performance venue publication.

3. Bajracharya, S. “AI-Driven Threat Detection for Kubernetes-Based Cloud Systems.” IJSRCSEIT, 2026, 12(3), 182–191.

4. Yang, Y., et al. “Securing the AI Agent: A Unified Framework for Multi-Layer Agent Red Teaming.” arXiv:2606.31227, 2026. (AI-Infra-Guard)

5. CNCF Falco documentation; Kubernetes documentation; Ollama documentation; ChromaDB documentation; k3s documentation.

6. Terragoat (Bridgecrew/Palo Alto) and CloudGoat (Rhino Security Labs) for intentionally vulnerable IaC/cloud practice environments.

Web-verified items used for this deck: SARGE • active behavioral validation • Kubernetes AI telemetry • AI-Infra-Guard.